Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы оптимизации и управления»

ОТЧЕТ
к лабораторной работе №2
на тему:
«ПРИВЕДЕНИЕ ЗАДАЧИ ЛИНЕЙНОГО ПРОГРАММИРОВАНИЯ К НОРМАЛЬНОЙ И КАНОНИЧЕСКОЙ ФОРМАМ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил доцент каф. ИКТ
ГОРБУКОВ Дмитрий Михайлович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Реализовать методы приведения задачи линейного программирования к нормальной и канонической формам. На вход подаются вектор коэффициентов целевой функции, тип оптимизации, матрица ограничений, правая часть, знаки ограничений и ограничения на знаки переменных.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Реализованы две функции: to_normal_form и to_canonical_form. Первая приводит задачу к виду max при ограничениях типа <= и неотрицательных переменных. Вторая приводит задачу к каноническому виду max, Ax = b, x >= 0 с добавлением дополнительных переменных. Для демонстрации использован пример из методических указаний:

Исходная задача линейного программирования:

```
-x1 + x2 - 2x3 -> min
x1 + x3 <= 1
x1 - x3 >= 2
x1 + x2 = 10
x1 >= 0, x2 <= 0, x3 unrestricted
```

При переходе к нормальной форме выполняются следующие преобразования: смена направления оптимизации при необходимости, замена переменных с ограничением x_j <= 0, представление свободной переменной как разности двух неотрицательных и преобразование равенств и ограничений вида >= к ограничениям типа <=.

Программа формирует следующую нормальную форму задачи:

```
A = [
  [1, 0, 1, -1]
  [-1, 0, 1, -1]
  [1, -1, 0, 0]
  [-1, 1, 0, 0]
]
b = [1, -2, 10, -10]
r = [<=, <=, <=, <=]
sigma = [>= 0, >= 0, >= 0, >= 0]
```

При переходе к канонической форме дополнительно вводятся переменные, чтобы все ограничения стали равенствами. Для рассматриваемого примера получается следующий результат:

```
A = [
  [1, 0, 1, -1, 1, 0]
  [1, 0, -1, 1, 0, -1]
  [1, -1, 0, 0, 0, 0]
]
b = [1, 2, 10]
r = [=, =, =]
sigma = [>= 0, >= 0, >= 0, >= 0, >= 0, >= 0]
```

Результат работы программы приведён ниже на рисунке 2.1.

```
Целевая функция: -1.0x1 + 1.0x2 + -2.0x3 -> min
Ограничения:
  (1.0)x1 + (0.0)x2 + (1.0)x3 <= 1.0
  (1.0)x1 + (0.0)x2 + (-1.0)x3 >= 2.0
  (1.0)x1 + (1.0)x2 + (0.0)x3 = 10.0
Переменные:
  x1 >= 0
  x2 <= 0
  x3 unrestricted
--------------------
--- Нормальная форма ---
Целевая функция: 1.0x1 + 1.0x2 + 2.0x3 + -2.0x4 -> max
Ограничения:
  (1.0)x1 + (-0.0)x2 + (1.0)x3 + (-1.0)x4 <= 1.0
  (-1.0)x1 + (0.0)x2 + (1.0)x3 + (-1.0)x4 <= -2.0
  (1.0)x1 + (-1.0)x2 + (0.0)x3 + (-0.0)x4 <= 10.0
  (-1.0)x1 + (1.0)x2 + (-0.0)x3 + (0.0)x4 <= -10.0
Переменные:
  x1 >= 0
  x2 >= 0
  x3 >= 0
  x4 >= 0
--------------------
--- Каноническая форма ---
Целевая функция: 1.0x1 + 1.0x2 + 2.0x3 + -2.0x4 + 0.0x5 + 0.0x6 -> max
Ограничения:
  (1.0)x1 + (-0.0)x2 + (1.0)x3 + (-1.0)x4 + (1.0)x5 + (0.0)x6 = 1.0
  (1.0)x1 + (-0.0)x2 + (-1.0)x3 + (1.0)x4 + (0.0)x5 + (-1.0)x6 = 2.0
  (1.0)x1 + (-1.0)x2 + (0.0)x3 + (-0.0)x4 + (0.0)x5 + (0.0)x6 = 10.0
Переменные:
  x1 >= 0
  x2 >= 0
  x3 >= 0
  x4 >= 0
  x5 >= 0
  x6 >= 0
```

Рисунок 2.1 – Результат работы программы

ВЫВОД

В лабораторной работе реализованы универсальные процедуры перехода от общей постановки задачи линейного программирования к нормальной и канонической формам. На рассмотренном примере программа корректно изменила направление оптимизации, обработала ограничения на знаки переменных и автоматически добавила новые переменные. Полученные результаты совпадают с преобразованиями, приведенными в методических указаниях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Python Documentation [Электронный ресурс]. – Режим доступа: https://docs.python.org/. – Дата доступа: 09.04.2026.

[2] NumPy Documentation [Электронный ресурс]. – Режим доступа: https://numpy.org/doc/. – Дата доступа: 09.04.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация приведения задачи ЛП к нормальной и канонической формам

```
import numpy as np
from copy import deepcopy

def print_lp_problem(c, d, A, b, r, sigma, title="Задача ЛП"):
    """Выводит задачу линейного программирования в читаемом виде."""
    print(f"--- {title} ---")

    # Целевая функция
    func_str = " + ".join([f"{c[i]}x{i+1}" for i in range(len(c))])
    print(f"Целевая функция: {func_str} -> {d}")

    # Ограничения
    print("Ограничения:")
    for i in range(A.shape[0]):
        row_str = " + ".join([f"({A[i, j]})x{j+1}" for j in
range(A.shape[1])])
        print(f"  {row_str} {r[i]} {b[i]}")

    # Ограничения на переменные
    print("Переменные:")
    for i in range(len(sigma)):
        print(f"  x{i+1} {sigma[i]}")
    print("--------------------")


def to_normal_form(c, d, A, b, r, sigma):
    """
    Приводит задачу ЛП к нормальной форме.
    Нормальная форма:
    - c'x -> max
    - A'x <= b'
    - x >= 0
    """
    c, d, A, b, r, sigma = deepcopy(c), deepcopy(d), deepcopy(A),
deepcopy(b), deepcopy(r), deepcopy(sigma)

    # Шаг 1: Целевая функция на max
    if d == 'min':
        c = -c
        d = 'max'

    # Шаг 2: Преобразование переменных
    num_vars = A.shape[1]
    for j in range(num_vars):
        # Переменные <= 0
        if sigma[j] == '<= 0':
            c[j] = -c[j]
            A[:, j] = -A[:, j]
            sigma[j] = '>= 0'
```

```
        # Переменные без ограничения знака
        elif sigma[j] == 'unrestricted':
            # Заменяем x_j на x_j' - x_j''
            A = np.insert(A, j + 1, -A[:, j], axis=1)
            c = np.insert(c, j + 1, -c[j])
            sigma[j] = '>= 0'
            sigma.insert(j + 1, '>= 0')

    num_constraints = A.shape[0]
    i = 0
    while i < num_constraints:
        # Шаг 3: Ограничения-равенства
        if r[i] == '=':
            A = np.vstack([A, A[i]])
            b = np.append(b, b[i])
            r[i] = '<='
            r.insert(i + 1, '>=')
            num_constraints += 1

        # Шаг 4: Ограничения >=
        elif r[i] == '>=':
            A[i, :] = -A[i, :]
            b[i] = -b[i]
            r[i] = '<='
        i += 1

    return c, d, A, b, r, sigma


def to_canonical_form(c, d, A, b, r, sigma):
    """
    Приводит задачу ЛП к канонической форме.
    Каноническая форма:
    - c'x -> max
    - A'x = b'
    - x >= 0
    """
    c, d, A, b, r, sigma = deepcopy(c), deepcopy(d), deepcopy(A),
deepcopy(b), deepcopy(r), deepcopy(sigma)

    # Шаг 1: Целевая функция на max
    if d == 'min':
        c = -c
        d = 'max'

    # Шаг 2: Преобразование переменных (аналогично нормальной форме)
    num_vars = A.shape[1]
    j = 0
    while j < num_vars:
        if sigma[j] == '<= 0':
            c[j] = -c[j]
            A[:, j] = -A[:, j]
            sigma[j] = '>= 0'
        elif sigma[j] == 'unrestricted':
            A = np.insert(A, j + 1, -A[:, j], axis=1)
            c = np.insert(c, j + 1, -c[j])
            sigma[j] = '>= 0'
            sigma.insert(j + 1, '>= 0')
```

```
            num_vars +=1
        j += 1

    # Шаг 3: Преобразование ограничений
    num_constraints = A.shape[0]
    for i in range(num_constraints):
        # b_i < 0
        if b[i] < 0:
            A[i,:] *= -1
            b[i] *= -1
            if r[i] == '<=': r[i] = '>='
            elif r[i] == '>=': r[i] = '<='

    for i in range(num_constraints):
        if r[i] == '<=':
            A = np.insert(A, A.shape[1], 0, axis=1)
            A[i, -1] = 1
            c = np.append(c, 0)
            sigma.append('>= 0')
            r[i] = '='
        elif r[i] == '>=':
            A = np.insert(A, A.shape[1], 0, axis=1)
            A[i, -1] = -1
            c = np.append(c, 0)
            sigma.append('>= 0')
            r[i] = '='

    return c, d, A, b, r, sigma


if __name__ == "__main__":
    c_initial = np.array([-1.0, 1.0, -2.0])
    d_initial = 'min'
    A_initial = np.array([
        [1.0, 0.0, 1.0],
        [1.0, 0.0, -1.0],
        [1.0, 1.0, 0.0]
    ])
    b_initial = np.array([1.0, 2.0, 10.0])
    r_initial = ['<=', '>=', '=']
    sigma_initial = ['>= 0', '<= 0', 'unrestricted']

    print_lp_problem(c_initial, d_initial, A_initial, b_initial,
r_initial, sigma_initial, "Исходная задача (3)")

    # Приведение к нормальной форме
    c_norm, d_norm, A_norm, b_norm, r_norm, sigma_norm =
to_normal_form(c_initial, d_initial, A_initial, b_initial, r_initial,
sigma_initial)
    print_lp_problem(c_norm, d_norm, A_norm, b_norm, r_norm,
sigma_norm, "Нормальная форма")

    # Приведение к канонической форме
    c_canon, d_canon, A_canon, b_canon, r_canon, sigma_canon =
to_canonical_form(c_initial, d_initial, A_initial, b_initial,
r_initial, sigma_initial)
    print_lp_problem(c_canon, d_canon, A_canon, b_canon, r_canon,
sigma_canon, "Каноническая форма")
```